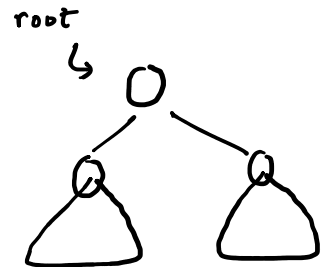
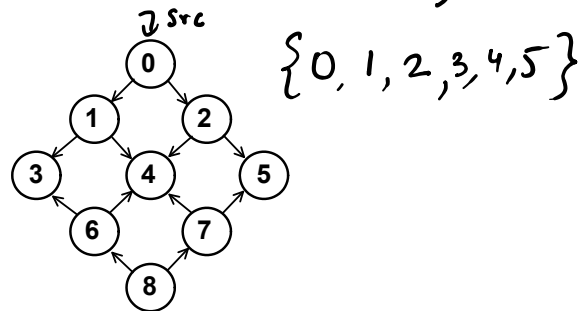
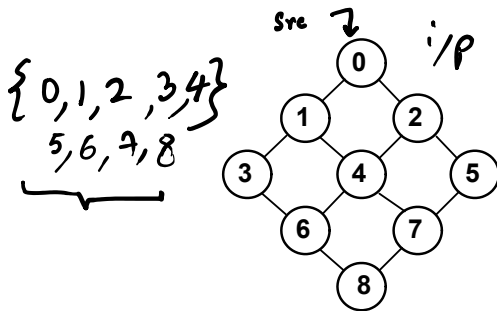


dfs bfs

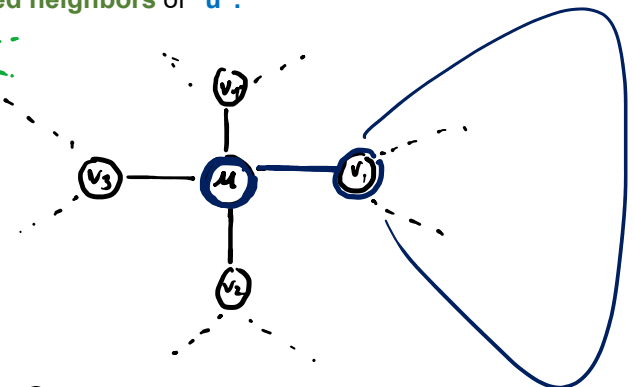
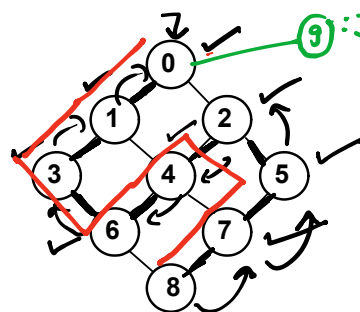
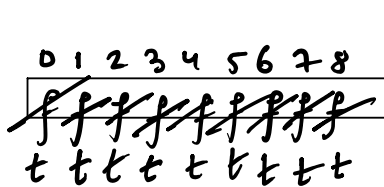
Reachability in Graphs

Given a **connected** graph and a **source** vertex, design an algorithm to find all the vertices **reachable** from the **source** vertex. A vertex "**v**" in the graph is said to be **reachable** from vertex "**u**" if there exists a **path** from vertex "**u**" to vertex "**v**".



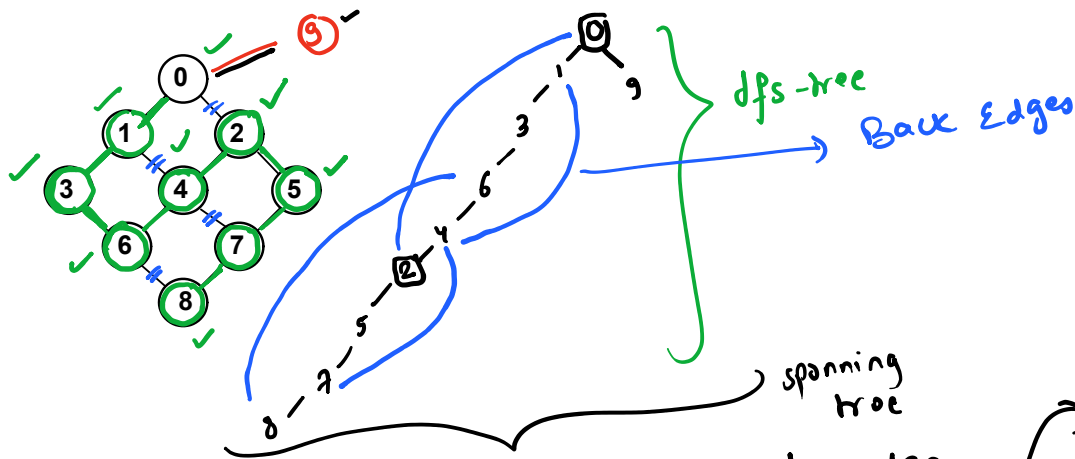
Depth First Search (DFS)

The key idea behind the **DFS algorithm** is that for any vertex "**u**" in the graph, when you visit one of its **unvisited neighbors**, say "**v**", then first you visit all the **unvisited vertices reachable** from "**v**" before you visit the other **unvisited neighbors** of "**u**".



0 1 3 6 4 2 5 7 8

$0 \rightarrow [1, 2]$
 $1 \rightarrow [0, 3, 4]$
 $2 \rightarrow [0, 4, 5]$
 $3 \rightarrow [1, 6]$
 $4 \rightarrow [1, 2, 6, 7]$
 $5 \rightarrow [2, 7]$
 $6 \rightarrow [3, 4, 8]$
 $7 \rightarrow [4, 5, 8]$
 $8 \rightarrow [6, 7]$



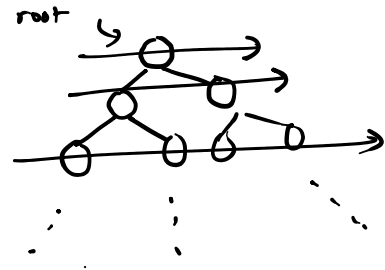
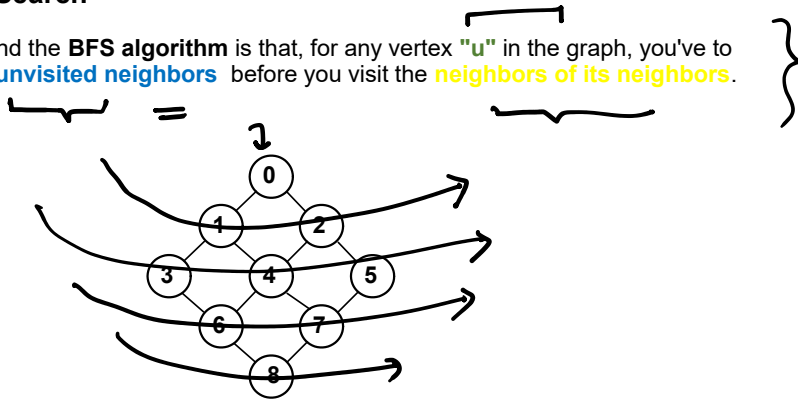
* If a graph has a backedge then it means gr. has a cycle

an edge present in the gr. but not in the spanning tree built by dfs/bfs

* o/p of dfs is not unique
— it depends on how you process the nodelist.

Breadth First Search

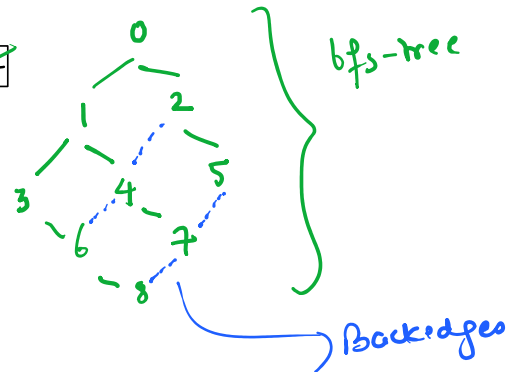
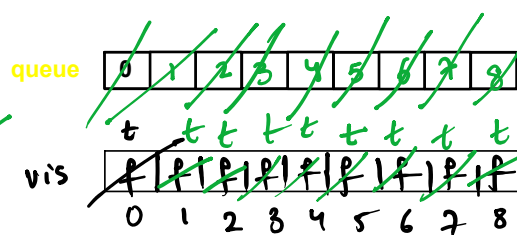
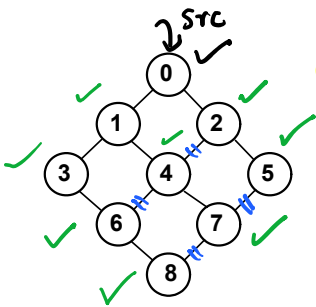
The key idea behind the **BFS algorithm** is that, for any vertex "u" in the graph, you've to first visit all of its **unvisited neighbors** before you visit the **neighbors of its neighbors**.

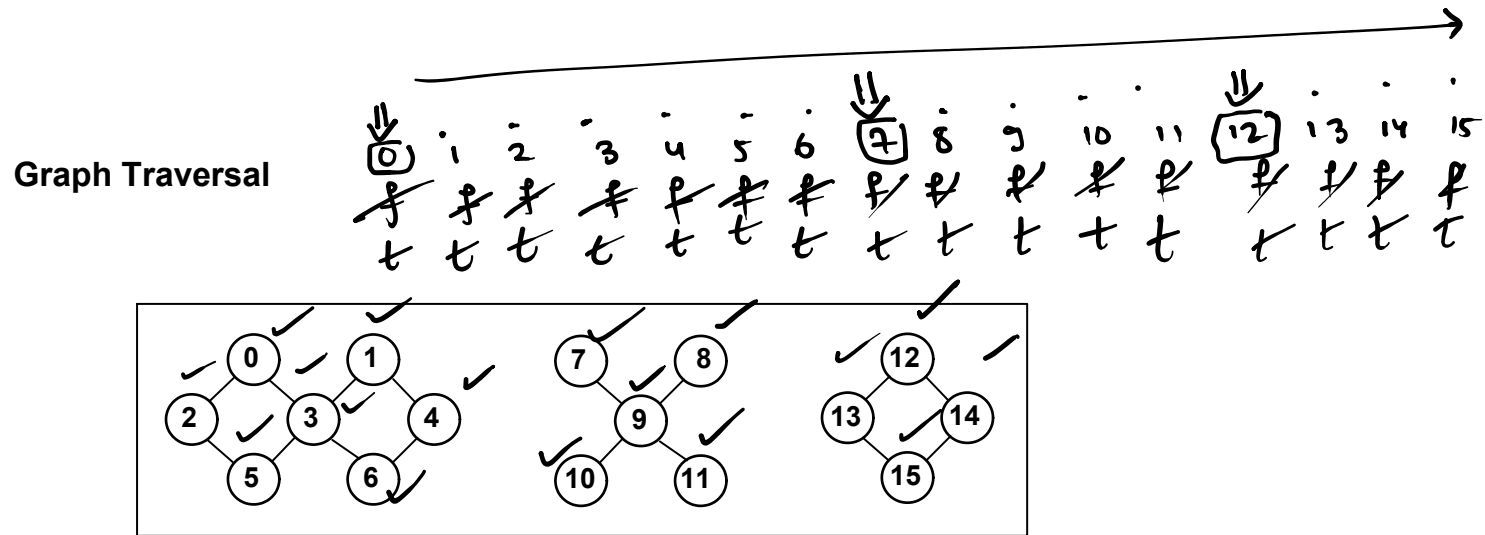


BFS Implementation

To implement the BFS algorithm, we will maintain

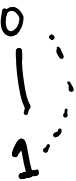
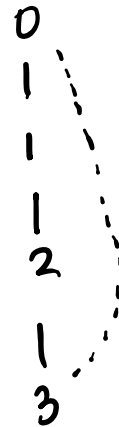
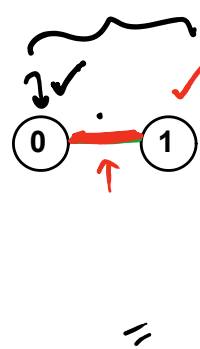
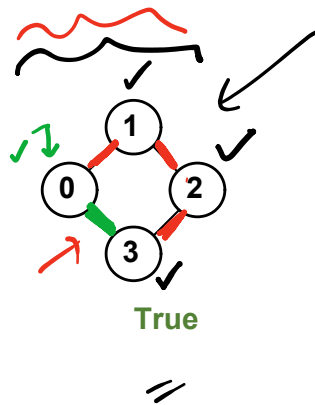
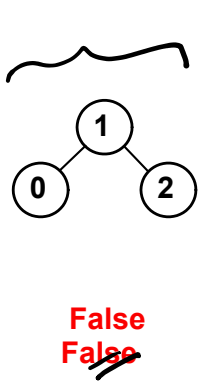
- a **Queue** to keep track of nodes that have been **visited** but not yet **explored**.
- a **Set / Map** to keep track of nodes that have been **visited**.





Cycle Detection

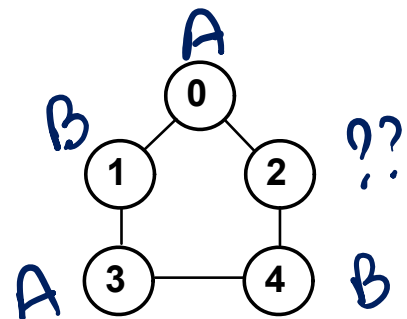
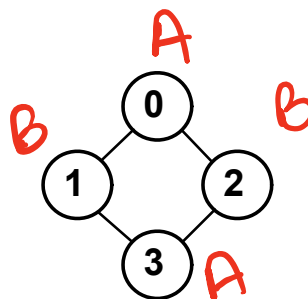
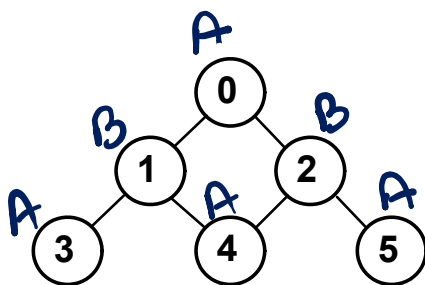
Given an **undirected** graph design an algorithm to check if contains a **cycle**.



Bipartite Graph 2-coloring Problem

Given an **undirected** graph design an algorithm to check if it is a **Bipartite** graph.

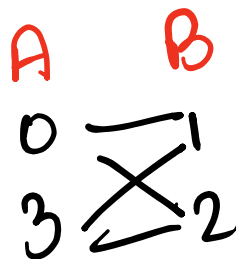
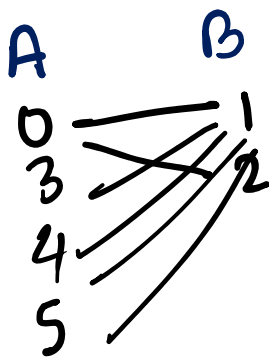
A graph is said to be a **Bipartite** graph if the graph nodes can be **partitioned** into **two independent sets** A and B such that each edge in the graph connects a node in set A and a node in set B.



True
False

True

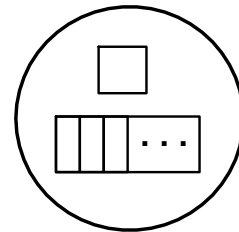
=



Clone Graph

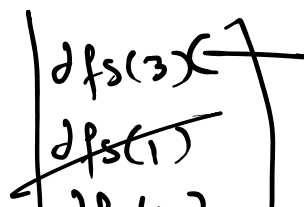
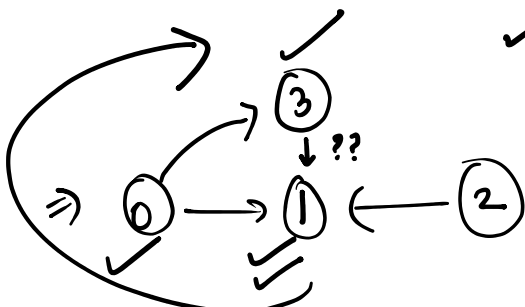
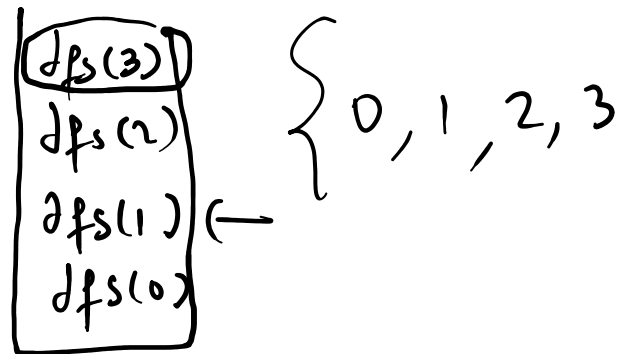
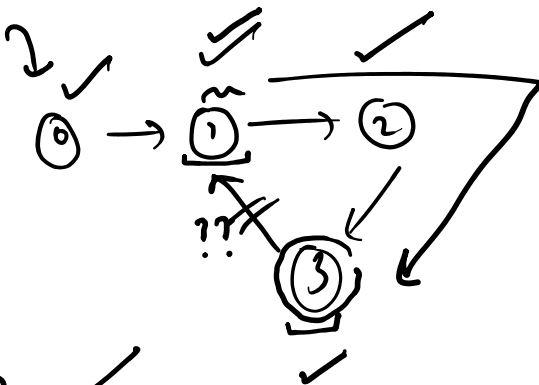
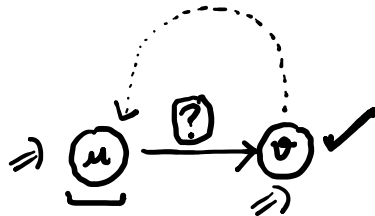
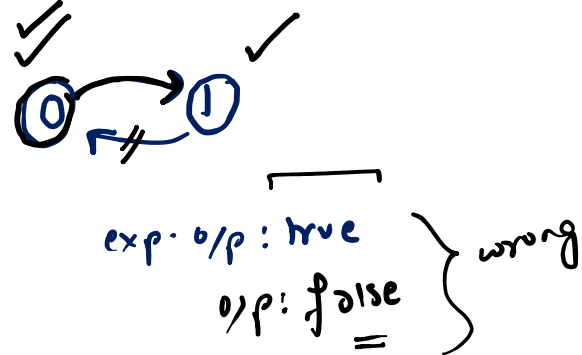
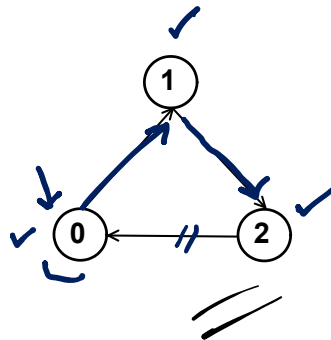
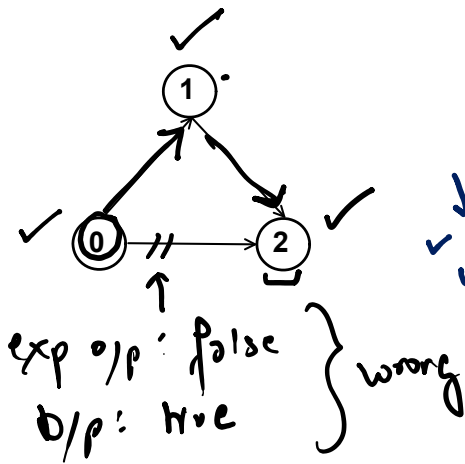
Given reference of a node in a connected undirected graph, return a **deep copy** (clone) of the graph. Note that, each graph node contains a value (*int*) and a list (*List [Node]*) of its neighbors.

```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```



Back Edge Detection

Given an **directed** graph design an algorithm to check if contains a **back-edge**.
 A directed edge $u \rightarrow v$ is said to be a back-edge if there exists a **directed path** from vertex v to vertex u .





~~$\partial p_s(1)$~~
 $\partial p_s(10)$